

PROJECT COMPLETION REPORT

INFOSYS SPRINGBOARD VIRTUAL INTERNSHIP 6.0

Project Title - Tempest FWI Predictor

Name: AKSHAY BAIJU

Batch No : 7

Section 1: Dataset Overview

1.1 Dataset Introduction

The Fire Weather Index (FWI) dataset contains daily meteorological observations and fire fuel indices collected from different regions in Algeria, used to predict forest fire weather conditions.

1.2 Feature Description

The dataset includes the following 15 features:

Temporal Features:

- **Day:** Day of the month (1-31)
- **Month:** Month of the year (1-12)
- **Year:** Year of observation

Meteorological Features:

- **Temperature (°C):** Ambient air temperature
- **Relative Humidity (%):** Moisture content in air
- **Wind Speed (km/h):** Speed of wind
- **Rain (mm):** Rainfall amount

Fire Fuel Indices:

- **FFMC (Fine Fuel Moisture Code):** Moisture content of surface litter
- **DMC (Duff Moisture Code):** Moisture content of organic layers
- **DC (Drought Code):** Long-term moisture deficiency in deep soil
- **ISI (Initial Spread Index):** Rate of fire spread after ignition
- **BUI (Buildup Index):** Total fuel available for combustion

Geographical Feature:

- **Region:** Two regions - Bejaia and Sidi Bel Abbes
- **Class:** Fire/Not Fire classification (0 or 1)

Target Variable:

- **FWI (Fire Weather Index):** Continuous numerical value to be predicted

Section 2: Data Preprocessing and Exploratory Data Analysis

2.1 Data Cleaning Steps

The following data cleaning operations were performed on the raw dataset:

Step 1: Loading and Column Standardization

- Loaded dataset from CSV file
- Converted all column names to lowercase
- Stripped whitespace from column values
- Total records before cleaning: 244 observations

Step 2: Data Type Conversion

- Converted numerical columns (day, month, year, temperature, rh, ws, rain, ffmc, dmc, dc, isi, bui, fwi) to numeric type
- Removed rows with non-numeric values in numerical columns
- Retained rows where numeric conversion was successful

Step 3: Missing Value Handling

- Identified all null values across the dataset
- Applied mean imputation for missing numerical values
- Missing values were replaced with the column mean to preserve distribution

Step 4: Duplicate Removal

- Identified and removed duplicate rows from the dataset
- Ensured each observation is unique

Step 5: Region Assignment

- First 122 records assigned to “bejaia” region
- Remaining records assigned to “sidi_bel_abbes” region
- Added region as a new categorical feature

Step 6: Fire Classification Encoding

- Original “classes” column contained text values (“Fire” and “Not Fire”)
- Converted to numerical: “fire” = 1, “not fire” = 0
- Created new “class” column with binary values
- Dropped original “classes” column

Step 7: Final Validation

- Verified no remaining null values in numerical columns
- Confirmed all data types are correct
- Saved cleaned dataset as “Algeria_dataset_clean.csv”

Final dataset: 244 records with 15 features

Section 3: Exploratory Data Analysis (EDA)

3.1 Histogram Analysis

Histograms were created to visualize distribution of features across different value ranges:

Temperature Distribution:

- Bins: <25°C, 25-30°C, 30-35°C, >35°C
- Analyzed average FWI for each temperature range
- Higher temperatures corresponded to higher average FWI values

Relative Humidity Distribution:

- Bins: <30%, 30-50%, 50-70%, >70%
- Observed inverse relationship between humidity and FWI
- Lower humidity associated with higher FWI

Wind Speed Distribution:

- Bins: 0-10 km/h, 10-20 km/h, >20 km/h
- Higher wind speeds showed increased average FWI

Rainfall Distribution:

- Bins: 0mm, 0-5mm, >5mm
- Rainfall showed negative correlation with FWI

FFMC Distribution:

- Bins: <60, 60-75, 75-85, >85
- Higher FFMC values associated with increased FWI

DMC Distribution:

- Bins: <10, 10-25, 25-50, >50
- Progressive increase in average FWI with higher DMC

DC Distribution:

- Bins: <50, 50-100, 100-150, >150

- Higher DC values linked to higher FWI

ISI Distribution:

- Bins: <2, 2-5, 5-10, >10
- Strong positive relationship between ISI and FWI

BUI Distribution:

- Bins: <20, 20-40, 40-60, >60
- Positive correlation with FWI observed across ranges

All histogram plots displayed with proper spacing and labeled axes for clarity.

3.2 Correlation Analysis

A correlation heatmap was generated to examine relationships between features and FWI:

Strong Positive Correlations with FWI:

- FPMC (Fine Fuel Moisture Code)
- ISI (Initial Spread Index)
- Temperature
- BUI (Buildup Index)

Negative Correlations with FWI:

- Rain (negative correlation - rainfall reduces fire index)
- Relative Humidity (negative correlation - moisture reduces fire risk)

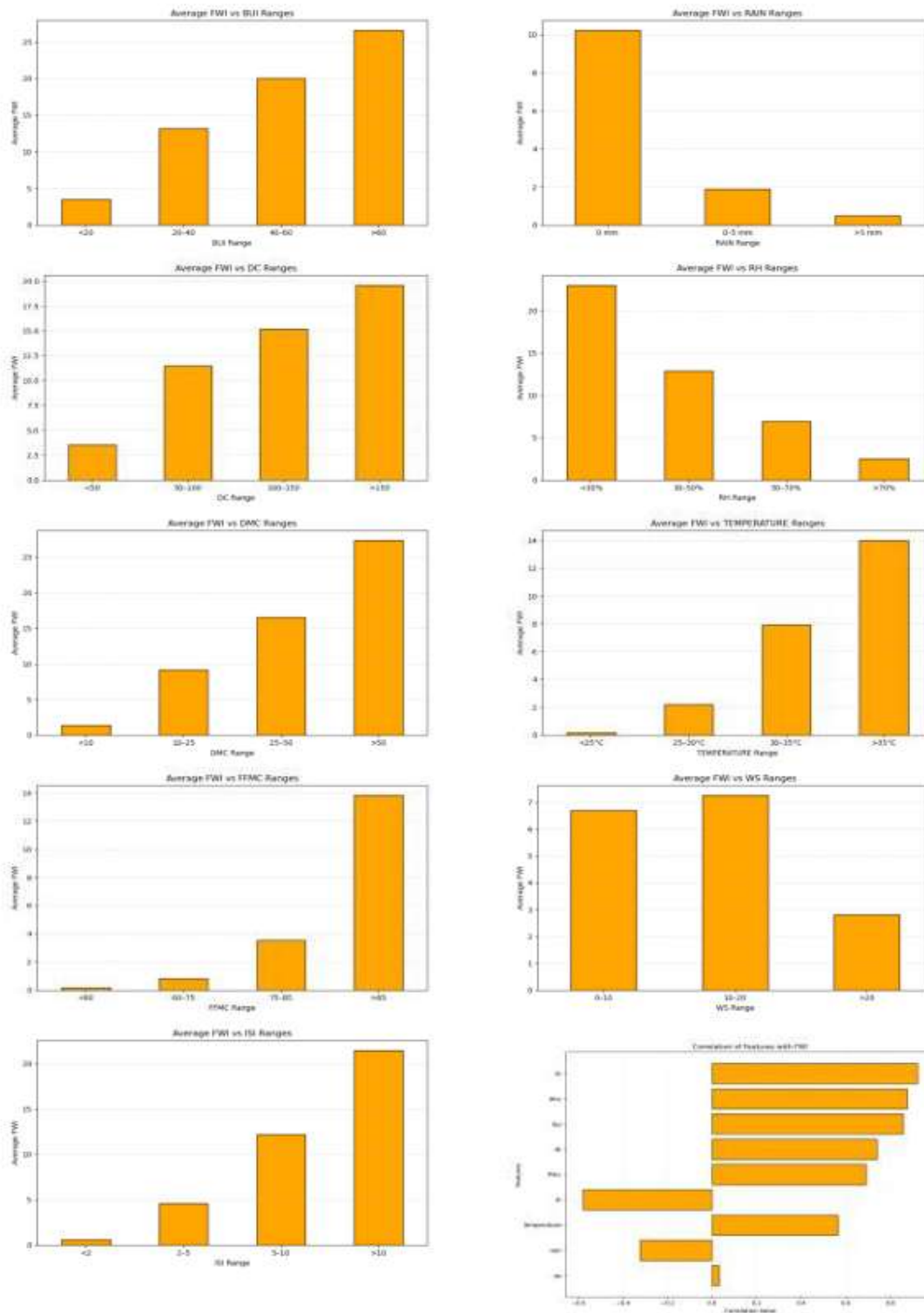
Moderate Correlations:

- DMC, DC, Wind Speed show moderate relationships with FWI

Inter-feature Correlations:

- FPMC and DMC show correlation
- Temperature and moisture indices show relationships
- Year-to-year and seasonal variations identified

The correlation matrix was computed using only numerical columns, excluding categorical data. Sorted features by absolute correlation magnitude to identify strongest relationships with target variable.



EDA PLOTS – Histograms and Corelation

Section 4: Feature Selection and Encoding

4.1 Feature Selection

Selected input features for the model:

- Temperature, Relative Humidity, Wind Speed, Rain (meteorological variables)
- FFMCI, DMC, DC, ISI, BUI (fire fuel indices)
- Region (geographical feature)
- Day, Month, Year (temporal features)

Selected target variable: **FWI** (Fire Weather Index)

Excluded from modeling:

- “Class” column (fire/no-fire binary label - not used as target for regression)

4.2 Categorical Feature Encoding

Region Encoding:

- Converted Region column to categorical dtype
- Bejaia encoded as category code 0
- Sidi_bel_abbes encoded as category code 1
- Categorical values converted to numeric form for regression model

Section 5: Data Splitting

5.1 Train-Test Split

Dataset partitioning:

- **Training Set:** 80% of data (195 records) - used to train the model
- **Test Set:** 20% of data (49 records) - used to evaluate model performance
- **Random State:** Set to 42 for reproducibility

This 80-20 split ensures the model is trained on sufficient data while retaining an adequate test set for unbiased performance evaluation.

Section 6: Feature Scaling

6.1 Standardization Using StandardScaler

Features had different units and value ranges (temperature in °C, indices ranging from 0-100+), requiring standardization.

Scaling Formula Applied:

$$X_scaled = (X - \mu) / \sigma$$

Where:

- X = original feature values
- μ = mean of training data
- σ = standard deviation of training data

Implementation Details:

1. Fitted StandardScaler on training data only
2. Calculated mean and standard deviation from X_train
3. Transformed training data: $X_train_scaled = (X_train - mean) / std$
4. Transformed test data using training parameters: $X_test_scaled = (X_test - mean) / std$
5. Saved fitted scaler to "scaler.pkl" for deployment

Post-Standardization Properties:

- Mean of each feature = 0
- Standard deviation of each feature = 1

6.2 Data Preprocessing Pseudocode

BEGIN

Load Algeria_dataset_clean.csv

Identify temporal features: day, month, year

Identify meteorological features: temperature, rh, ws, rain

Identify fuel indices: ffmc, dmc, dc, isi, bui

Identify geographical feature: region

Identify target: fwi

Encode Region (categorical) to numeric codes

SPLIT data into 80% train, 20% test

X_train, X_test = training and test features

y_train, y_test = training and test targets

INITIALIZE StandardScaler

Fit scaler on X_train :

Calculate mean of each feature from X_train

Calculate std of each feature from X_train

Scale X_train :

$X_train_scaled = (X_train - mean) / std$

Scale X_test using training parameters:

$X_{\text{test_scaled}} = (X_{\text{test}} - \text{mean}) / \text{std}$

SAVE scaler to scaler.pkl

RETURN X_train_scaled, X_test_scaled, y_train, y_test

END

Section 7: Ridge Regression Model Development

7.1 Ridge Regression Implementation

Ridge Regression applies L2 regularization to handle feature multicollinearity and prevent overfitting.

7.2 Hyperparameter Tuning

Alpha Values Tested: [0.0001, 0.001, 0.1, 0.5, 1, 2, 5, 10, 20, 50, 75, 100]

For each alpha value:

1. Created Ridge Regression model with specified alpha
2. Trained model on X_train_scaled and y_train
3. Generated predictions on X_train_scaled and X_test_scaled
4. Calculated performance metrics for both sets

7.3 Performance Metrics Calculated

For each alpha value, computed:

Training Set Metrics:

- MSE (Mean Squared Error)
- RMSE (Root Mean Squared Error)
- MAE (Mean Absolute Error)
- R^2 Score

Test Set Metrics:

- MSE (Mean Squared Error)
- RMSE (Root Mean Squared Error)
- MAE (Mean Absolute Error)
- R^2 Score

7.4 Best Model Selection

Identified optimal alpha based on minimum test MSE. The alpha value with lowest test error was selected to ensure best generalization on unseen data.

7.5 Final Model Training

Retrained Ridge Regression model using best alpha on complete training dataset:

- Used optimal alpha value identified from tuning
- Trained on full `X_train_scaled` with `y_train`
- Generated final predictions on `X_test_scaled`
- Calculated final performance metrics

7.6 Model Visualization and Evaluation

Actual vs Predicted Plots:

- Linear Regression: Scatter plot of predicted vs actual FWI values
- Ridge Regression: Scatter plot of predicted vs actual FWI values
- Both plots included diagonal reference line for perfect prediction
- Plots showed prediction accuracy and error distribution

Error Analysis Plots:

- MAE vs Alpha: Line plot showing how MAE changes with alpha values
- RMSE vs Alpha: Line plot showing RMSE variation across alphas
- Identified optimal alpha from these visualizations

Overfitting Check:

- Compared training MSE with test MSE
- Ensured no significant difference (no overfitting)
- Verified model generalizes well to unseen data

7.7 Model Persistence

Saved trained Ridge Regression model to "ridge_model.pkl" containing:

- Learned coefficients for all features
- Intercept value
- Model parameters and hyperparameters

7.8 Ridge Regression Training Pseudocode

BEGIN

Load `X_train_scaled`, `X_test_scaled`, `y_train`, `y_test`

DEFINE `alpha_values` = [0.0001, 0.001, 0.1, 0.5, 1, 2, 5, 10, 20, 50, 75, 100]

Initialize lists for storing results:

```
alpha_mse_scores = []  
alpha_mae_scores = []
```

```
FOR each alpha in alpha_values DO  
    CREATE Ridge model with current alpha
```

```
    TRAIN on X_train_scaled, y_train
```

```
    PREDICT on X_train_scaled:  
        y_train_pred = model.predict(X_train_scaled)  
    PREDICT on X_test_scaled:  
        y_test_pred = model.predict(X_test_scaled)
```

```
    CALCULATE training metrics:  
        train_mse = mean((y_train - y_train_pred)2)  
        train_rmse = sqrt(train_mse)  
        train_mae = mean(|y_train - y_train_pred|)  
        train_r2 = score(y_train, y_train_pred)
```

```
    CALCULATE test metrics:  
        test_mse = mean((y_test - y_test_pred)2)  
        test_rmse = sqrt(test_mse)  
        test_mae = mean(|y_test - y_test_pred|)  
        test_r2 = score(y_test, y_test_pred)
```

```
    STORE test_mse in alpha_mse_scores  
    STORE test_mae in alpha_mae_scores  
END FOR
```

```
FIND best_alpha with minimum test_mse
```

```
CREATE final Ridge model with best_alpha  
TRAIN on X_train_scaled, y_train
```

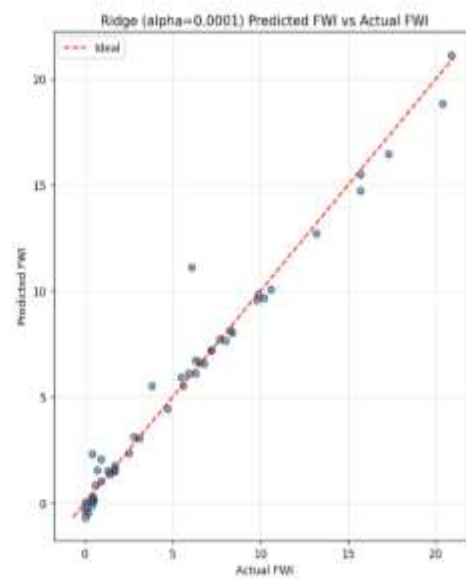
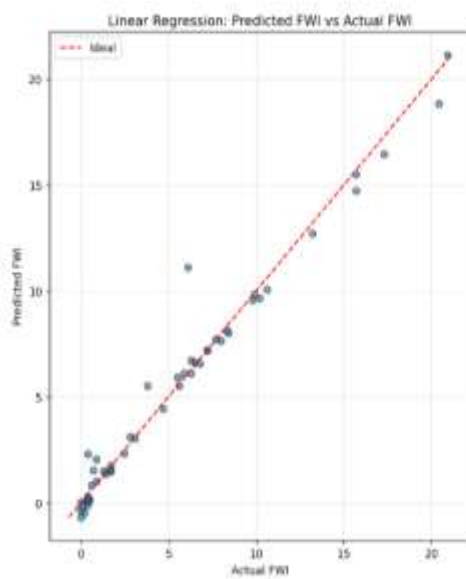
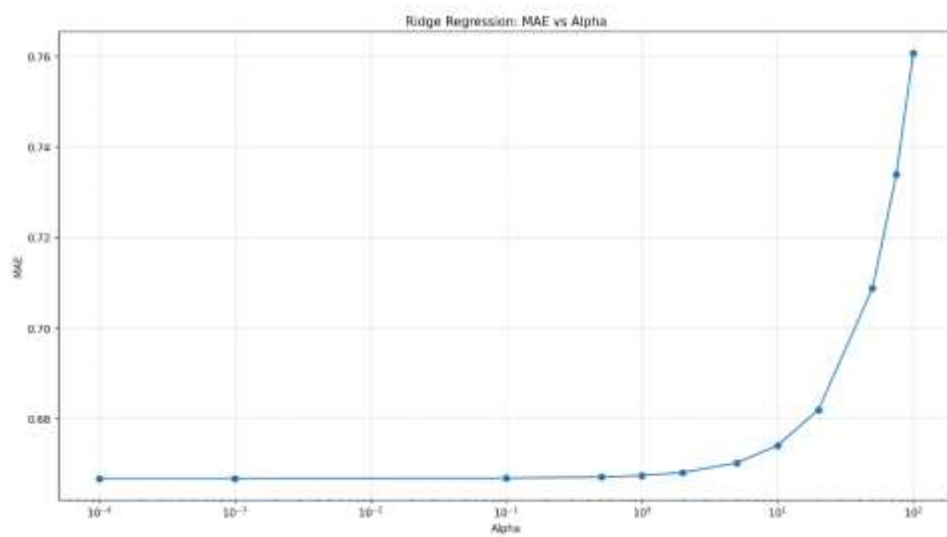
```
PREDICT final results:  
    y_train_final = model.predict(X_train_scaled)  
    y_test_final = model.predict(X_test_scaled)
```

```
CALCULATE final metrics for both sets
```

```
PLOT:  
    - Actual vs Predicted (scatter plot)  
    - Alpha vs MAE (line plot)  
    - Alpha vs RMSE (line plot)
```

```
SAVE model to ridge_model.pkl
```

```
RETURN trained_model, best_alpha, final_metrics
END
```



Section 8: Flask Web Application

8.1 Application Overview

Flask web application deployed to integrate the trained Ridge Regression model with an interactive dashboard for real-time FWI predictions.

8.2 Model Loading

At application startup:

1. Check for existence of ridge_model.pkl file
2. Check for existence of scaler.pkl file
3. Load both files into memory if present
4. Create temporary demo models if files not found
5. Store loaded model and scaler for prediction use

8.3 User Interface

Dashboard Input Form Fields:

- Temperature (numeric input)
- Relative Humidity (numeric input)
- Wind Speed (numeric input)
- Rainfall (numeric input)
- FFMC (numeric input)
- DMC (numeric input)
- DC (numeric input)
- ISI (numeric input)
- BUI (numeric input)
- Region (dropdown selection)

Form Design:

- HTML form with labeled input fields
- Bootstrap styling for responsive layout
- Submit button to trigger prediction
- Clear labels for each input parameter

8.4 Prediction Process

Step 1: Input Reception

- Receive form data via HTTP POST request
- Extract values: temperature, rh, ws, rain, dmc, ffmc, dc, isi, bui, region

Step 2: Data Preparation

- Arrange features in exact order used during training

- Create DataFrame with proper column names

Step 3: Feature Scaling

- Load scaler.pkl
- Apply scaling: $(\text{value} - \text{mean}) / \text{std}$

Step 4: Model Prediction

- Load ridge_model.pkl
- Pass scaled features to model
- Model outputs predicted FWI value

Step 5: Risk Categorization

- Compare FWI with thresholds:
 - FWI ≥ 12.0 : HIGH RISK (🔥 High Chance of Fire)
 - $6.0 \leq \text{FWI} < 12.0$: MODERATE RISK (⚠️ Moderate Fire Chance)
 - FWI < 6.0 : LOW RISK (✓ Low Fire Chance)

Step 6: Response Display

- Return prediction results to frontend
- Display predicted FWI value (rounded to 3 decimals)
- Display risk level with visual indicator
- Show threshold information

8.5 Application Execution

Server Details:

- Run on local address: `http://127.0.0.1:5000/`
- Development server mode enabled (`debug=True`)
- No external tunneling required
- Accessible via web browser on development machine

Routes Implemented:

Route 1: GET /

- Returns dashboard HTML template
- Displays input form and prediction interface

Route 2: POST /predict

- Receives user input from form
- Processes prediction
- Returns JSON response with FWI and risk level
- Handles errors gracefully with error messages

8.6 Error Handling

- Try-catch blocks around prediction logic
- Handle missing input values
- Handle invalid input types
- Return user-friendly error messages
- Maintain application stability on errors

8.7 Flask Application Pseudocode

BEGIN

INITIALIZE Flask application

LOAD ridge_model.pkl

LOAD scaler.pkl

DEFINE risk thresholds:

 HIGH_RISK_THRESHOLD = 12.0

 MODERATE_RISK_THRESHOLD = 6.0

CREATE route "/" (GET):

 RETURN render_template("dashboard.html")

CREATE route "/predict" (POST):

 TRY

 EXTRACT from form:

 temperature, rh, ws, rain

 dmc, ffmc, dc, isi, bui, region

 ARRANGE in feature order:

 [temperature, rh, ws, rain, dmc, ffmc, dc, isi, bui, region]

 CREATE input DataFrame with these values

 SCALE input using loaded scaler:

 input_scaled = (input - scaler.mean) / scaler.std

 PREDICT using loaded model:

 predicted_fwi = ridge_model.predict(input_scaled)

 DETERMINE risk level:

 IF predicted_fwi >= 12.0:

 risk_level = "high"

 ELSE IF predicted_fwi >= 6.0:

 risk_level = "moderate"

 ELSE:

```
risk_level = "low"
```

```
APPEND to prediction history
```

```
UPDATE chart visualization
```

```
RETURN JSON response:
```

```
{  
  "fwi": round(predicted_fwi, 3),  
  "risk_level": risk_level,  
  "threshold_high": 12.0,  
  "threshold_moderate": 6.0  
}
```

```
CATCH Exception as e:
```

```
RETURN error response with error message
```

```
END TRY
```

```
RUN app on 127.0.0.1:5000
```

```
END
```

Flask App screenshots: -

The screenshot displays the 'Fire Weather Index Predictor' web application. At the top, there is a logo of a flame inside a circle, followed by the title 'Fire Weather Index Predictor' and the subtitle 'Advanced fire risk assessment powered by machine learning'. Below this, a green button labeled '🔍 Single Request Mode' is visible. The main form is divided into two sections: 'Atmospheric Conditions' and 'Fuel Moisture Codes'. The 'Atmospheric Conditions' section includes input fields for Temperature (°C) with a value of 35, Relative Humidity (%) with a value of 57, and Wind Speed (km/h) with a value of 15. Below these are Range and Unit dropdown menus. The 'Fuel Moisture Codes' section includes input fields for FMC (Fuel Moisture Code) with a value of 95.3, DMC (Duff Moisture Code) with a value of 23.5, and EMC (Ember Moisture Code) with a value of 16.3. There is also an input field for ISI (Initial Spread Index) with a value of 11.40. At the bottom right, a green button labeled 'Calculate Fire Risk' is present. A small disclaimer at the bottom left states: 'All values are maximum values! Input values for training data for model also predicted here.'

High Chance of Fire

Fire Weather Alert

17.3

Fire Weather Index • High Chance of Fire (FWI > 10.0)
Fire Weather Index: 17.300000



Extreme Fire Danger

The Fire Weather Index is in the High-risk band (FWI > 10.0).
Conditions are highly favorable for fire ignition and rapid spread. Avoid any open flames, follow local fire bans, and be prepared to act quickly if a fire is reported nearby.

0 New Prediction

Section 9: Conclusion

The Tempest FWI Predictor successfully achieves its objective of predicting Fire Weather Index values using machine learning. The project demonstrates comprehensive understanding of the machine learning pipeline including data preprocessing, exploratory analysis, model development, optimization, and deployment. The resulting Flask web application provides an accessible tool for fire weather prediction that could support forest fire prevention and management efforts in fire-prone regions.

This project serves as a complete example of how machine learning can be applied to environmental and safety challenges, translating scientific understanding into practical tools that benefit end users.